



图 9-53 一棵已分配块的平衡树中的左右指针

平衡树方法保证会标记所有从根节点可达的节点，从这个意义上来说它是正确的。这是一个必要的保证，因为应用程序的用户当然不会喜欢把他们的已分配块过早地返回给空闲链表。然而，这种方法从某种意义上而言又是保守的，因为它可能不正确地标记实际上不可达的块，因此它可能不会释放某些垃圾。虽然这并不影响应用程序的正确性，但是这可能导致不必要的外部碎片。

C 程序的 Mark & Sweep 收集器必须是保守的，其根本原因是 C 语言不会用类型信息来标记内存位置。因此，像 `int` 或者 `float` 这样的标量可以伪装成指针。例如，假设某个可达的已分配块在它的有效载荷中包含一个 `int`，其值碰巧对应于某个其他已分配块 `b` 的有效载荷中的一个地址。对收集器而言，是没有办法推断出这个数据实际上是 `int` 而不是指针。因此，分配器必须保守地将块 `b` 标记为可达，尽管事实上它可能是不可达的。

9.11 C 程序中常见的与内存有关的错误

对 C 程序员来说，管理和使用虚拟内存可能是个困难的、容易出错的任务。与内存有关的错误属于那些最令人惊恐的错误，因为它们在时间和空间上，经常在距错误源一段距离之后才表现出来。将错误的数据写到错误的位置，你的程序可能在最终失败之前运行了好几个小时，且使程序中止的位置距离错误的位置已经很远了。我们用一些常见的与内存有关错误的讨论，来结束对虚拟内存的讨论。

9.11.1 间接引用坏指针

正如我们在 9.7.2 节中学到的，在进程的虚拟地址空间中有较大的洞，没有映射到任何有意义的数据。如果我们试图间接引用一个指向这些洞的指针，那么操作系统就会以段异常中止程序。而且，虚拟内存的某些区域是只读的。试图写这些区域将会以保护异常中止这个程序。

间接引用坏指针的一个常见示例是经典的 `scanf` 错误。假设我们想要使用 `scanf` 从 `stdin` 读一个整数到一个变量。正确的方法是传递给 `scanf` 一个格式串和变量的地址：

```
scanf("%d", &val)
```

然而，对于 C 程序员初学者而言(对有经验者也是如此!)，很容易传递 `val` 的内容，而不是它的地址：

```
scanf("%d", val)
```

在这种情况下，`scanf` 将把 `val` 的内容解释为一个地址，并试图将一个字写到这个位置。在最好的情况下，程序立即以异常终止。在最糟糕的情况下，`val` 的内容对应于虚拟内存的某个合法的读/写区域，于是我们就覆盖了这块内存，这通常会在相当长的一段时间以后造成灾难性的、令人困惑的后果。

9.11.2 读未初始化的内存

虽然 `bss` 内存位置(诸如未初始化的全局 C 变量)总是被加载器初始化为零，但是对于堆内存却并不是这样的。一个常见的错误就是假设堆内存被初始化为零：

```

1  /* Return y = Ax */
2  int *matvec(int **A, int *x, int n)
3  {
4      int i, j;
5
6      int *y = (int *)Malloc(n * sizeof(int));
7
8      for (i = 0; i < n; i++)
9          for (j = 0; j < n; j++)
10             y[i] += A[i][j] * x[j];
11     return y;
12 }

```

在这个示例中，程序员不正确地假设向量 y 被初始化为零。正确的实现方式是显式地将 $y[i]$ 设置为零，或者使用 `calloc`。

9.11.3 允许栈缓冲区溢出

正如我们在 3.10.3 节中看到的，如果一个程序不检查输入串的大小就写入栈中的目标缓冲区，那么这个程序就会有缓冲区溢出错误(buffer overflow bug)。例如，下面的函数就有缓冲区溢出错误，因为 `gets` 函数复制一个任意长度的串到缓冲区。为了纠正这个错误，我们必须使用 `fgets` 函数，这个函数限制了输入串的大小：

```

1  void bufoverflow()
2  {
3      char buf[64];
4
5      gets(buf); /* Here is the stack buffer overflow bug */
6      return;
7  }

```

9.11.4 假设指针和它们指向的对象是相同大小的

一种常见的错误是假设指向对象的指针和它们所指向的对象是相同大小的：

```

1  /* Create an nxm array */
2  int **makeArray1(int n, int m)
3  {
4      int i;
5      int **A = (int **)Malloc(n * sizeof(int));
6
7      for (i = 0; i < n; i++)
8          A[i] = (int *)Malloc(m * sizeof(int));
9      return A;
10 }

```

这里的目的是创建一个由 n 个指针组成的数组，每个指针都指向一个包含 m 个 `int` 的数组。然而，因为程序员在第 5 行将 `sizeof(int *)` 写成了 `sizeof(int)`，代码实际上创建的是一个 `int` 的数组。

这段代码只有在 `int` 和指向 `int` 的指针大小相同的机器上运行良好。但是，如果我们在像 Core i7 这样的机器上运行这段代码，其中指针大于 `int`，那么第 7 行和第 8 行的循环将

写到超出 A 数组结尾的地方。因为这些字中的一个很可能是已分配块的边界标记脚部，所以我们可能不会发现这个错误，直到在这个程序的后面很久释放这个块时，此时，分配器中的合并代码会戏剧性地失败，而没有任何明显的原因。这是“在远处起作用(action at distance)”的一个阴险的示例，这类“在远处起作用”是与内存有关的编程错误的典型情况。

9.11.5 造成错位错误

错位(off-by-one)错误是另一种很常见的造成覆盖错误的来源：

```

1  /* Create an nxm array */
2  int **makeArray2(int n, int m)
3  {
4      int i;
5      int **A = (int **)Malloc(n * sizeof(int *));
6
7      for (i = 0; i <= n; i++)
8          A[i] = (int *)Malloc(m * sizeof(int));
9      return A;
10 }
```

这是前面一节中程序的另一个版本。这里我们在第 5 行创建了一个 n 个元素的指针数组，但是随后在第 7 行和第 8 行试图初始化这个数组的 $n+1$ 个元素，在这个过程中覆盖了 A 数组后面的某个内存位置。

9.11.6 引用指针，而不是它所指向的对象

如果不太注意 C 操作符的优先级和结合性，我们会错误地操作指针，而不是指针所指向的对象。比如，考虑下面的函数，其目的是删除一个有 $*size$ 项的二叉堆里的第一项，然后对剩下的 $*size-1$ 项重新建堆：

```

1  int *binheapDelete(int **binheap, int *size)
2  {
3      int *packet = binheap[0];
4
5      binheap[0] = binheap[*size - 1];
6      *size--; /* This should be (*size)-- */
7      heapify(binheap, *size, 0);
8      return(packet);
9  }
```

在第 6 行，目的是减少 $size$ 指针指向的整数的值。然而，因为一元运算符 $--$ 和 $*$ 的优先级相同，从右向左结合，所以第 6 行中的代码实际减少的是指针自己的值，而不是它所指向的整数的值。如果幸运地话，程序会立即失败；但是更有可能发生的是，当程序在执行过程后很久才产生出一个不正确的结果时，我们只有一头的雾水。这里的原则是当你对优先级和结合性有疑问的时候，就使用括号。比如，在第 6 行，我们可以使用表达式 $(*size)--$ ，清晰地表明我们的意图。

9.11.7 误解指针运算

另一种常见的错误是忘记了指针的算术操作是以它们指向的对象的大小为单位来进行的，而这种大小单位并不一定是字节。例如，下面函数的目的是扫描一个 int 的数组，并

返回一个指针，指向 val 的首次出现：

```

1  int *search(int *p, int val)
2  {
3      while (*p && *p != val)
4          p += sizeof(int); /* Should be p++ */
5      return p;
6  }
```

然而，因为每次循环时，第 4 行都把指针加了 4(一个整数的字节数)，函数就不正确地扫描数组中每 4 个整数。

9.11.8 引用不存在的变量

没有太多经验的 C 程序员不理解栈的规则，有时会引用不再合法的本地变量，如下列所示：

```

1  int *stackref ()
2  {
3      int val;
4
5      return &val;
6  }
```

这个函数返回一个指针(比如说是 p)，指向栈里的一个局部变量，然后弹出它的栈帧。尽管 p 仍然指向一个合法的内存地址，但是它已经不再指向一个合法的变量了。当以后在程序中调用其他函数时，内存将重用它们的栈帧。再后来，如果程序分配某个值给 *p，那么它可能实际上正在修改另一个函数的栈帧中的一个条目，从而潜在地带来灾难性的、令人困惑的后果。

9.11.9 引用空闲堆块中的数据

一个相似的错误是引用已经被释放了的堆块中的数据。例如，考虑下面的示例，这个示例在第 6 行分配了一个整数数组 x，在第 10 行中先释放了块 x，然后在第 14 行中又引用了它：

```

1  int *heapref(int n, int m)
2  {
3      int i;
4      int *x, *y;
5
6      x = (int *)Malloc(n * sizeof(int));
7
8      : // Other calls to malloc and free go here
9
10     free(x);
11
12     y = (int *)Malloc(m * sizeof(int));
13     for (i = 0; i < m; i++)
14         y[i] = x[i]++; /* Oops! x[i] is a word in a free block */
15
16     return y;
17 }
```


取决于在第 6 行和第 10 行发生的 `malloc` 和 `free` 的调用模式，当程序在第 14 行引用 `x[i]` 时，数组 `x` 可能是某个其他已分配堆块的一部分了，因此其内容被重写了。和其他许多与内存有关的错误一样，这个错误只会在程序执行的后面，当我们注意到 `y` 中的值被破坏了时才会显现出来。

9.11.10 引起内存泄漏

内存泄漏是缓慢、隐性的杀手，当程序员不小心忘记释放已分配块，而在堆里创建了垃圾时，会发生这种问题。例如，下面的函数分配了一个堆块 `x`，然后不释放它就返回：

```
1 void leak(int n)
2 {
3     int *x = (int *)Malloc(n * sizeof(int));
4
5     return; /* x is garbage at this point */
6 }
```

如果经常调用 `leak`，那么渐渐地，堆里就会充满了垃圾，最糟糕的情况下，会占用整个虚拟地址空间。对于像守护进程和服务器这样的程序来说，内存泄漏是特别严重的，根据定义这些程序是不会终止的。

9.12 小结

虚拟内存是对主存的一个抽象。支持虚拟内存的处理器通过使用一种叫做虚拟寻址的间接形式来引用主存。处理器产生一个虚拟地址，在被发送到主存之前，这个地址被翻译成一个物理地址。从虚拟地址空间到物理地址空间的地址翻译要求硬件和软件紧密合作。专门的硬件通过使用页表来翻译虚拟地址，而页表的内容是由操作系统提供的。

虚拟内存提供三个重要的功能。第一，它为主存中自动缓存最近使用的存放磁盘上的虚拟地址空间的内容。虚拟内存缓存中的块叫做页。对磁盘上页的引用会触发缺页，缺页将控制转移到操作系统中的一个缺页处理程序。缺页处理程序将页面从磁盘复制到主存缓存，如果必要，将写回被驱逐的页。第二，虚拟内存简化了内存管理，进而又简化了链接、在进程间共享数据、进程的内存分配以及程序加载。最后，虚拟内存通过在每条页表条目中加入保护位，从而简化了内存保护。

地址翻译的过程必须和系统中所有的硬件缓存的操作集成在一起。大多数页表条目位于 L1 高速缓存中，但是一个称为 TLB 的页表条目的片上高速缓存，通常会消除访问在 L1 上的页表条目的开销。

现代系统通过将虚拟内存片和磁盘上的文件片关联起来，来初始化虚拟内存片，这个过程称为内存映射。内存映射为共享数据、创建新的进程以及加载程序提供了一种高效的机制。应用可以使用 `mmap` 函数来手工地创建和删除虚拟地址空间的区域。然而，大多数程序依赖于动态内存分配器，例如 `malloc`，它管理虚拟地址空间区域内一个称为堆的区域。动态内存分配器是一个感觉像系统级程序的应用级程序，它直接操作内存，而无需类型系统的很多帮助。分配器有两种类型。显式分配器要求应用显式地释放它们的内存块。隐式分配器（垃圾收集器）自动释放任何未使用的和不可达的块。

对于 C 程序员来说，管理和使用虚拟内存是一件困难和容易出错的任务。常见的错误示例包括：间接引用坏指针，读取未初始化的内存，允许栈缓冲区溢出，假设指针和它们指向的对象大小相同，引用指针而不是它所指向的对象，误解指针运算，引用不存在的变量，以及引起内存泄漏。

参考文献说明

Kilburn 和他的同事们发表了第一篇关于虚拟内存的描述[63]。体系结构教科书包括关于硬件在虚拟内存中的角色的更多细节[46]。操作系统教科书包含关于操作系统角色的更多信息[102, 106, 113]。Bovet 和 Cesati [11]给出了 Linux 虚拟内存系统的详细描述。Intel 公司提供了 IA 处理器上 32 位和 64 位